

BÀI THỰC HÀNH 1:
LINUX CÁC LỆNH CƠ BẢN

1 Giới thiệu về bộ thông dịch lệnh bash 2
1.1 Trình thông dịch lệnh là gì ? 2
1.2 Kiểm tra có phải sử dụng bash shell không? 2
2 Các lệnh cơ bản của Linux 3
2.1 Lệnh chuyển thư mục "cd" 3
2.1.1 Đường dẫn (Paths) trong hệ điều hành Linux 3
2.1.2 Đường dẫn tuyệt đối (Absolute paths) 3
2.1.3 Đường dẫn tương đối (Relative paths) 3
2.1.4 Sử dụng thư mục cha ".." 3
2.1.5 Một số ví dụ về đường dẫn tương đối 4
2.1.6 Thư mục hiện hành "." 4
2.1.7 Chuyển về thư mục cá nhân (home directory) 4
2.1.8 Đường dẫn đến thư mục cá nhân của người dùng bất kỳ 4
2.2 Giới thiệu lệnh "ls" 5
2.2.1 Xem thông tin về thư mục 6
2.2.2 Xem nội dung của tất cả các thư mục con 6
2.2.3 Inodes 6
2.3 Lệnh tạo thư mục mkdir 7
2.4 Lệnh tạo tập tin rỗng (touch) 7
2.5 Hiển thị thông tin và định hướng lại 8
2.6 Lệnh cat và cp 8
2.7 Lệnh mv 8
3 Tạo các liên kết và xóa các tập tin 8
3.1 Liên kết cứng (Hard links) 8
3.2 Liên kết mềm (Symbolic links) 9
3.3 Lệnh xóa tập tin rm 10
3.4 Lệnh xóa thư mục rmdir 11
4 Sử dụng ký tự đại diện (wildcards) 11
4.1 Sử dụng ký tự * 12
4.2 Sử dụng ký tự ? 12
4.3 Ký tự [] 12
4.4 Ký tự [!] 12
4.5 Wildcard caveats 12

1 Giới thiệu về bộ thông dịch lệnh bash

1.1 Trình thông dịch lệnh là gì ?

Dưới Linux, sau khi đăng nhập (login) vào hệ thống dưới chế độ văn bản (text) hoặc mở một cửa sổ lệnh (terminal) trong chế độ đồ họa (graphics) , bạn được chào đón bằng một **Dấu nhắc** (prompt) như sau:

$

Dạng thức của **Dấu nhắc** có thể thay đổi tùy từng hệ thống. Nó có thể bao gồm cả tên của máy tính (hostname), thư mục hiện hành đang làm việc của bạn (current working directory). Bất kể dạng thức của dấu nhắc như thế nào, một điều chắc chắn là bạn đang giao tiếp với một chương trình được gọi là **Trình thông dịch lệnh** (shell).

Trình thông dịch lệnh được dùng phổ biến trên hệ thống Linux là chương trình **bash** (Viết tắt của cụm từ Bourne-Again Shell).

1.2 Kiểm tra có phải sử dụng bash shell không?

Bạn có thể kiểm tra xem hệ thống bạn đang sử dụng có dùng bash shell không bằng lệnh sau:

```
$ echo $SHELL
/bin/bash
```

Nếu kết quả trả về không đúng như trên mà xuất hiện một thông báo lỗi, điều đó cho thấy hệ thống của bạn đang chạy một shell khác.

Bash là shell mặc định trên hầu hết các hệ điều hành Linux. Nhiệm vụ của một chương trình shell là thực hiện các lệnh do bạn nhập vào, giúp bạn tương tác được với hệ điều hành Linux.

Bạn có thể kết thúc chương trình shell khi đã hoàn thành phiên làm việc bằng lệnh **exit** hoặc **logout**, hay nhấn tổ hợp phím Control-D tại dấu nhắc của bash.

2 Các lệnh cơ bản của Linux

2.1 Lệnh chuyển thư mục "cd"

Mục đích: Lệnh cd cho phép bạn di chuyển đến những điểm khác nhau trên cây thư mục. Cú pháp tổng quát:

$cd path

Trong đó path là đường dẫn chỉ đến thư mục mà ta muốn chuyển đến. Ví dụ: Chuyển về thư mục gốc của cây thư mục ta đánh lệnh:

$ cd /

2.1.1 Đường dẫn (Paths) trong hệ điều hành Linux

Để xem thư mục làm việc hiện hành của bạn, hãy nhập vào lệnh sau: $ pwd /

Lệnh trên cho thấy bạn đang ở thư mục gốc (ký hiệu bằng ký tự '/') của cây thư mục.
Có hai loại đường dẫn: Đường dẫn tuyệt đối và đường dẫn tương đối.

2.1.2 Đường dẫn tuyệt đối (Absolute paths)

Đường dẫn tuyệt đối được bắt đầu bằng ký tự /. Ví dụ:

/dev
/usr
/usr/bin
/usr/local/bin

Bạn có thể chuyển vào thư mục /user/loal/bin bằng lệnh sau, từ bất kỳ thư mục hiện hành nào:

cd /usr/local/bin

2.1.3 Đường dẫn tương đối (Relative paths)

Đường dẫn tương đối được tính toán với điểm bắt đầu của đường dẫn được ngầm định là thư mục hiện hành của bạn. Một đường dẫn tương đối không bao giờ bắt đầu với ký tự '/'.

Ví dụ:

Để chuyển vào thư mục /usr bạn sử dụng lệnh sau:

$ cd /usr

Lúc này thư mục hiện hành của bạn là /usr. Ở đó bạn có thể chuyển vào thư mục /usr/local/bin bằng cách sử dụng đường dẫn tương đối như sau:

$ cd local/bin
$ pwd
/usr/local/bin

2.1.4 Sử dụng thư mục cha ".."

Thư mục cha của thư mục hiện hành được ký hiệu bằng chuỗi hai dấu chấm "..". Hãy thực hiện các lệnh sau: $ pwd
/usr/local/bin
$ cd ..

$ pwd
/usr/local
Như vậy, thư mục hiện hành hiện nay là /usr/local.
Bạn có thể sử dụng thư mục cha trong đường dẫn tương đối, ví dụ:
$ pwd
/usr/local
$ cd ../share $
pwd
/usr/share

2.1.5 Một số ví dụ về đường dẫn tương đối

$ cd /bin
$ cd ../usr/share/zoneinfo
$ cd /usr/X11R6/bin
$ cd ../lib/X11
$ cd /usr/bin
$ cd ../bin/../bin

2.1.6 Thư mục hiện hành "."

Thư mục hiện hành được ký hiệu bởi dấu chấm ".". Thư mục hiện hành thường được dùng để yêu cầu shell thực hiện một chương trình nào đó trong thư mục hiện hành.
Ví dụ:
$ ./myprog
Ví dụ trên yêu cầu shell thực thi chương trình myprog đang nằm trong thư mục hiện hành của bạn.

2.1.7 Chuyển về thư mục cá nhân (home directory)

Mỗi người dùng có một thư mục cá nhân để lưu trữ thông tin riêng của họ (gọi là home directory) nơi mà họ có toàn quyền trên đó trong khi những người khác thì không có quyền truy xuất đến. Bạn có thể chuyển về home directory của bạn từ bất kỳ vị trí nào trong cây thư mục bằng cách đánh lệnh cd không tham số:
$ cd
Khi đó bạn sẽ được chuyển về thư mục /home/username với username là tên đăng nhập của bạn.
Thư mục cá nhân được ký hiệu bằng ký tự '~'. Ví dụ ta thực hiện chương trình myprog trong thư mục hiện hành, với tham số là myfile.txt ở thư mục cá nhân bằng lệnh sau: $ ./myprog ~/myfile.txt

2.1.8 Đường dẫn đến thư mục cá nhân của người dùng bất kỳ

Bash dùng ký hiệu '~' để chỉ đến thư mục các nhân của bạn. Tuy nhiên bạn cũng có thể dùng nó để chỉ đến thư mục cá nhân của những người dùng khác. Ví dụ, bạn muốn tham khảo tập tin fredsfile.txt trong thư mục cá nhân của người dùng fred, bạn nhập vào dòng lệnh:
$ ./myprog ~fred/fredsfile.txt

2.2 Giới thiệu lệnh "ls"

Lệnh ls dùng để xem nội dung của một thư mục. Nhập vào các lệnh sau để xem nội dung của thư mục /usr:

```
$ cd /usr
$ ls
X11R6 doc i686-pc-linux-gnu lib man sbin ssl
bin gentoo-x86 include libexec portage share
tmp distfiles i686-linux info local portage.old
src
```

Nếu có thêm tham số -a, lệnh ls sẽ liệt kê luôn cả các tập tin ở dạng ẩn (tên tập tin bắt đầu bằng dấu chấm .):

```
$ ls -a
. bin gentoo-x86 include libexec portage share tmp
.. distfiles i686-linux info local portage.old src
X11R6 doc i686-pc-linux-gnu lib man sbin ssl
```

Nếu tham số **-l** được đưa vào, lệnh ls sẽ liệt kê nhiều thông tin hơn về các tập tin và thư mục nằm trong thư mục được mô tả, gồm các thông tin như: quyền truy cập tập tin, chủ sở hữu , ngày giờ cập nhật sau cùng, kích thước tập tin. Ví dụ

```
$ ls -l /usr
drwxr-xr-x 7 root root 168 Nov 24 14:02 X11R6 drwxr-
xr-x 2 root root 14576 Dec 27 08:56 bin drwxr-xr-x 2 root
root 8856 Dec 26 12:47 distfiles lrwxrwxrwx 1 root root
9 Dec 22 20:57 doc -> share/doc drwxr-xr-x 62 root root
1856 Dec 27 15:54 gentoo-x86 drwxr-xr-x 4 root root 152
Dec 12 23:10 i686-linux drwxr-xr-x 4 root root 96 Nov 24
13:17 i686-pc-linux-gnu drwxr-xr-x 54 root root 5992
Dec 24 22:30 include lrwxrwxrwx 1 root root 10 Dec 22
20:57 info -> share/info drwxr-xr-x 28 root root 13552
Dec 26 00:31 lib drwxr-xr-x 3 root root 72 Nov 25 00:34
libexec drwxr-xr-x 8 root root 240 Dec 22 20:57 local
lrwxrwxrwx 1 root root 9 Dec 22 20:57 man -> share/man
lrwxrwxrwx 1 root root 11 Dec 8 07:59 portage -> gentoo-
x86/ drwxr-xr-x 60 root root 1864 Dec 8 07:55 portage.old
drwxr-xr-x 3 root root 3096 Dec 22 20:57 sbin drwxr-xr-
x 46 root root 1144 Dec 24 15:32 share drwxr-xr-x 8 root
root 328 Dec 26 00:07 src drwxr-xr-x 6 root root 176 Nov
24 14:25 ssl lrwxrwxrwx 1 root root 10 Dec 22 20:57 tmp
-> ../var/tmp
```

Cột thứ nhất mô tả quyền truy cập đối tượng (tập tin hay thư mục)

Cột thứ hai mô tả số lượng các nối kết đến tập đối tượng Cột
thứ ba và tư mô tả chủ sở hữu và nhóm chủ sở hữu đối tượng.
Cột thứ năm mô tả kích thước đối tượng.
Cột thứ sáu mô tả ngày, giờ cập nhật đối tượng sau cùng.
Cuối cùng là tên của đối tượng. Nếu tên tập tin là một liên kết thì nó sẽ có dấu mũi tên "->" để chỉ đến đối tượng mà nó liên kết tới.

2.2.1 Xem thông tin về thư mục

Để chỉ xem thông tin về thư mục mà không cần thiết phải liệt kê nội dung của thư mục ta dùng lệnh ls với tham số là –dl. Ví dụ: $ ls -dl /usr /usr/bin /usr/X11R6/bin ../share drwxr-xr-x 4 root root 96 Dec 18 18:17 ../share drwxr-xr-x 17 root root 576 Dec 24 09:03 /usr drwxr-xr-x 2 root root 3192 Dec 26 12:52 /usr/X11R6/bin drwxr-xr-x 2 root root 14576 Dec 27 08:56 /usr/bin

2.2.2 Xem nội dung của tất cả các thư mục con

Sử dụng lệnh ls với tham số -R để xem nội dung của tất cả các thư mục con, của một thư mục.

2.2.3 Inodes

Mỗi đối tượng trong hệ thống tập tin của Linux được gán một số chỉ mục (index) duy nhất, được gọi là số inode. Bạn có thể xem thông tin về số inode của một tập tin hay thư mục bằng lệnh ls với tham số là –i. Ví dụ:

$ ls -id /usr/local
5120 /usr/local

Kết quả lệnh trên cho thấy thư mục /usr/local có số inode là 5120.
Tiếp tục xem inode của thư mục /usr/local/bin bằng lệnh sau:

$ ls -id /usr/local/bin/..
5120 /usr/local/bin/..

Ta thấy, /usr/local/bin/.. có cùng số inode với thư mục /usr/local. Điều đố có vẻ nghịch lý vì theo nguyên tắc mỗi thư mục hay tập tin được gán một inode riêng. Đây là cơ chế liên kết
(link) dưới Linux. Thực tế chỉ tồn tại một thư mục có số inode là 5120. Cả hai thư mục /usr/bin và /usr/local/bin đều liên kết đến cùng một thư mục có số inode là 5120. Như vậy cả hai thư mục trên có cùng một nội dung.
Dùng lệnh ls với tham số -dl để hiển thị nội dung chi tiết về một thư mục:

$ ls -dl /usr/local
drwxr-xr-x 8 root root 240 Dec 22 20:57 /usr/local

Tại cột thứ hai từ bên trái tính sang ta thấy có số 8 biểu thị rằng có 8 thư mục cùng liên kết đến inode số 5120. Trong hệ thống hiện tại là các thư mục sau (Có thể khác với hệ thống bạn đang sử dụng):

/usr/local
/usr/local/.
/usr/local/bin/..

/usr/local/games/..
/usr/local/lib/..
/usr/local/sbin/..
/usr/local/share/..
/usr/local/src/..

2.3 Lệnh tạo thư mục mkdir

Lệnh **mkdir** cho phép tạo một thư mục mới trong hệ thống tập tin. Ví dụ sau tạo ra 3 thư mục mới có tên là tic, tac và toe, trong thư mục /tmp:

$ cd /tmp

$ mkdir tic tac toe

Đánh lệnh $ls để kiểm tra xem hệ thống có tạo ra các thư mục tic, tac và toe không.

Mặc định, lệnh **mkdir** không tạo thư mục cha của thư mục mới khi chưa có. Vì thế nếu bạn muốn tạo thư mục won/der/ful bằng lệnh sau:

$ mkdir won/der/ful

Thì sẽ bị báo lỗi nếu một trong các thư mục won hay der chưa tồn tại:

mkdir: cannot create directory `won/der/ful': No such file or directory

$ mkdir won

$ mkdir won/der

$ mkdir won/der/ful

mkdir –p

Tuy nhiên bạn có thể yêu cầu lệnh mkdir tạo tất cả các thư mục cha chưa tồn tại bằng cách đưa thêm tùy chọn **-p**. Ví dụ dưới đây sẽ tạo ra 3 thư mục mới easy, as và pie theo thứ tự easy/as/pie:

$ mkdir -p easy/as/pie

2.4 Lệnh tạo tập tin rỗng (touch)

Cú pháp: touch fileName

Nếu tập tin đã tồn tại, touch sẽ cập nhật lại thời gian cập nhật sau cùng của tập tin là thời điểm thực hiện lệnh touch. Ngược lại, nếu tập tin chưa tồn tại, một tập tin rỗng được tạo ra. Ví dụ tạo file copyme trong thư mục /tmp:

$ cd /tmp

$ touch copyme

Dùng lệnh ls –l để xem lại thông tin về tập tin vừa được tạo ra. Chú ý đến ngày cập nhật sau cùng và kích thước của tập tin (Kích thước là 0 bytes).

Sau đó đánh lại lệnh touch:

$ touch copyme

Dùng lệnh ls –l để kiểm tra ngày cập nhật sau cùng của tập tin copyme đã được thay đổi so với lần trước chưa.

2.5 Hiển thị thông tin và định hướng lại

Lệnh echo dùng để xuất dữ liệu ra thiết bị xuất chuẩn (mặc định là màn hình). Cú pháp: echo “Data to export to standard ouput”. Ví dụ:

```
$ echo "firstfile"
firstfile
```

Tuy nhiên ta có thể chuyển hướng dữ liệu xuất ra các thiết bị xuất khác thiết bị xuất chuẩn bằng cách sử dụng cơ chế định hướng lại.

Cú pháp: command > newOutput

Ví du sau cho lệnh echo xuất dữ liệu ra tập tin copyme:

```
$ echo "firstfile" > copyme
```

Dùng lệnh ls để kiểm tra lại kích thước của tập tin copyme

```
$ ls -l copyme
-rw-r--r-- 1 root root 10 Dec 28 14:13 copyme
```

2.6 Lệnh cat và cp

Để hiển thị nội dung của một tập tin trên cửa sổ lệnh ta dùng lệnh **cat**.

Cú pháp: cat fileName

```
$ cat copyme
firstfile
```

Để sao chép một tập tin mới từ một tập tin đã có ta dùng lệnh **cp.**

Cú pháp: cp oldFile newFile

Ví dụ, chép tập tin mới copiedme từ tập tin copyme: $ cp copyme copiedme

Đây thật sự là 2 tập tin khác nhau với có số inode riêng biệt. Ta có thể kiểm tra bằng lệnh ls:

```
$ ls -i copyme copiedme 648284
copiedme 650704 copyme
```

2.7 Lệnh mv

Lệnh **mv** cho phép đổi tên một tập tin. Số hiệu inode vẫn được giữ lại, chỉ tên tập tin trỏ đến số hiệu inode đó bị thay đổi.

Ví dụ:

```
$ mv copiedme movedme
$ ls -i movedme
648284 movedme
```

3 Tạo các liên kết và xóa các tập tin

3.1 Liên kết cứng (Hard links)

Có hai loại liên kết trên Linux: Liên kết cứng và liên kết mềm. Một inode có thể có nhiều liên kết cứng nối với nó và inode sẽ được tồn tại đến khi nào tất cả các liên kết cứng nối với nó không còn nữa. Một liên kết mới có thể được tạo ra bằng cách sử dụng lệnh **ln** . Cú pháp : ln fileName newLink Ví dụ:

$ cd /tmp
$ touch firstlink
$ ln firstlink secondlink
$ ls -i firstlink secondlink
15782 firstlink 15782 secondlink
Hạn chế của Liên kết cứng là nó chỉ cho phép tạo các liên kết cứng đến các inode của tập tin. Liên kết cứng không cho phép chúng ta mở rộng hệ thống tập tin trên nhiều ổ đĩa. Có nghĩa là bạn không thể tạo một liên kết từ /usr/bin/bash đến /bin/bash nếu thư mục gốc của bạn / và thư mục /usr tồn tại trên 2 hệ thống tập tin riêng biệt.

3.2 Liên kết mềm (Symbolic links)

Trong thực tế liên kết mềm thường được dùng nhiều hơn liên kết cứng. Liên kết mềm là một kiểu tập tin đặc biệt mà ở đó liên kết tham khảo đến tập tin khác bằng tên thay vì tham khảo trục tiếp đến inode. Liên kết mềm không ngăn ngừa được trường hợp tập tin mà nó tham khảo đến đã bị xóa. Nếu tập tin đích không còn tồn tại, liên kết mềm xem như không còn sử dụng được hay hoàn toàn bị đỗ vỡ.
Một liên kết mềm có thể được tạo ra bằng lệnh **ln** với tùy chịn
là **–s**. Cú pháp: ln –s fileName newLink Ví dụ:
$ ln -s secondlink thirdlink
$ ls -l firstlink secondlink thirdlink
-rw-rw-r-- 2 agriffis agriffis 0 Dec 31 19:08 firstlink -
rw-rw-r-- 2 agriffis agriffis 0 Dec 31 19:08 secondlink
lrwxrwxrwx 1 agriffis agriffis 10 Dec 31 19:39 thirdlink -> secondlink
Liên kết mềm có thể phân biệt với các tập tin thường trong lệnh **ls -l** ở 3 đặc điểm trong cửa sổ kết quả:
Cột thứ nhất có chứa ký tự **l** để báo hiệu đó là một liên kết mềm.
Kích thước của liên kết mềm thì bằng với kích thước của tên tập tin đích (trong trường hợp này là tập tin **secondlink**).
Cột cuối cùng có hiển thị tên tập tin mà liên kết mềm trỏ đến.
Liên kết mềm thì mềm dẽo hơn liên kết cứng rất nhiều. Bạn có thể tạo một liên kết mềm đến bất kỳ kiểu đối tượng nào của hệ thống tập tin, bao gồm cả thư mục. Nó có thể trỏ đến một đối tượng trên một hệ thống tập tin khác.
Xét trường hợp mà ở đó ta muốn tạo một liên kết trong thư mục /tmp trỏ đến thư mục /usr/local/bin.
Hãy nhập vào các lệnh sau:
$ ln -s /usr/local/bin bin1
$ ls -l bin1
lrwxrwxrwx 1 root root 14 Jan 1 15:42 bin1 -> /usr/local/bin
Tương tự
$ ln -s ../usr/local/bin bin2
$ ls -l bin2

lrwxrwxrwx 1 root root 16 Jan 1 15:43 bin2 -> ../usr/local/bin
Ta thấy cả 2 liên kết mềm đều trỏ đến cùng một thư mục đích. Tuy nhiên nếu liên kết thứ hai được chuyển sang một thư mục khác thì nó sẽ vị gẩy ngay vì địa chỉ của tập tin đích được mô tả theo kiểu đường dẫn tương đối.
Hãy kiểm tra bằng các lếnh sau:
$ ls -l bin2
lrwxrwxrwx 1 root root 16 Jan 1 15:43 bin2 -> ../usr/local/bin
$ mkdir mynewdir
$ mv bin2 mynewdir
$ cd mynewdir
$ cd bin2
bash: cd: bin2: No such file or directory
Bởi vì thư mục /tmp/usr/local/bin không tồn tại, nên chúng ta không thể chuyển vào thư mục bin2, có nghĩa là liên kết bin2 đã bị "gẫy"
Vì lý do này thông thường người ta không dùng đường dẫn tương đối khi tạo liên kết mềm. Tuy nhiên cũng có trường hợp sử dụng đường dẫn tương đối trong liên kết mềm thì hợp lý hơn.
Xem ví dụ dưới đây, ở đó ta muốn tạo một tên mới cho một chương trình đang nằm trong thư mục /usr/bin: # ls -l /usr/bin/keychain
-rwxr-xr-x 1 root root 10150 Dec 12 20:09 /usr/bin/keychain
cd /usr/bin
ln -s /usr/bin/keychain kc
Giải pháp trên là đúng đắng, tuy nhiên sẽ có vấn đề ngay khi bạn quyết định chuyển cả hai tập tin vào thư mục /usr/local/bin:
mv /usr/bin/keychain /usr/bin/kc /usr/local/bin
Bởi vì sử dụng đường dẫn tuyệt đối trong liên kết mềm, **kc** vẫn tiếp tục trỏ đến /usr/bin/keychain, mà hiện tại nó không còn nữa. Như thế liên kết đã bị gẫy.
Cả đường dẫn tuyệt đối và tương đối sử dụng trong liên kết mềm đều có những điểm bất lợi riêng. Tùy tường trường hợp mà bạn chọn lựa loại đường dẫn cho phù hợp.

3.3 Lệnh xóa tập tin rm

Lệnh **rm** cho phép bạn xóa một tập tin ra khỏi hệ thống tập tin. Cú pháp: rm fileName Ví dụ:
$ cd /tmp
$ touch file1 file2
$ ls -l file1 file2
-rw-r--r-- 1 root root 0 Jan 1 16:41 file1
-rw-r--r-- 1 root root 0 Jan 1 16:41 file2
$ rm file1 file2
$ ls -l file1
file2

ls: file1: No such file or directory
ls: file2: No such file or directory

3.4 Lệnh xóa thư mục rmdir

Để xóa thư mục bạn có hai cách. Cách thứ nhất là xóa tất cả các đối tượng bên trong thư mục muốn xóa (làm cho thư mục trở nên rỗng). Sau đó dùng lệnh rmdir để xóa nó. Ví dụ:

$ mkdir mydir
$ touch mydir/file1
$ rm mydir/file1
$ rmdir mydir

Cách thứ hai là sử dụng lệnh **rm** với tùy chọn **–rf**.

Ví dụ

$ rm -rf mydir

4 Sử dụng ký tự đại diện (wildcards)

Trong khi sử dụng Linux, đôi khi bạn muốn thực hiện một tác vụ (ví dụ **rm**) trên nhiều đối tượng của hệ thống tập tin trên cùng một lượt. Trong trường hợp đó ta có thể dùng lệnh với tham số gồm nhiều tên tập tin.

Ví dụ:

$ rm file1 file2 file3 file4 file5 file6 file7 file8

Để giải quyết vấn đề này, bạn có thể sử dụng tiện ích hỗ trợ sẵn của Linux là các ký tự đại điện. Nó cho phép bạn chỉ ra nhiều tập tin một lượt bằng cách sử dụng một mẫu đại diện (*wildcard pattern)*. Trình thông dịch lệnh Bash và các trình thông dịch lệnh khác sẽ thông dịch mẫu bằng cách dò tìm trên đĩa những tập tin mà nó trùng khớp với mẫu được mô tả Giả sử bạn có các tập tin file1 đến file8 trong thư mục hiện hành, bạn có thể xóa cả 8 file này bằng lệnh sau: $ rm file[1-8]

Hay đơn giản hơn có thể mô tả rằng bạn muốn xóa tất cả các tập tin có tên mở đầu bằng file, hãy đánh lệnh sau: $ rm file*

Hoặc bạn muốn liệt kê tất cả các đối tượng trong hệ thống tập tin trong thư mục /etc có tên bắt đầu với ký tự **g**, bạn có thể gỏ lệnh sau:

$ ls -d /etc/g*
/etc/gconf /etc/ggi /etc/gimp /etc/gnome /etc/gnome-vfs-mime-magic /etc/gpm

Điều gì sẽ xảy ra nếu bạn mô tả một mẫu mà nó không trùng với bất kỳ đối tượng nào nằm trong hệ thống tập tin? Trong ví dụ sau, chúng muốn liệt kê tất cả các thư mục có trong thư mục /usr/bin mà chúng có tên bắt đầu với **asdf** và kết thúc với **jkl**:

$ ls -d /usr/bin/asdf*jkl
ls: /usr/bin/asdf*jkl: No such file or directory

Trong trường hợp này kết quả là một thông báo lỗi. Như vậy, với bash thì kết quả của các mẫu phải có ít nhất một trường hợp trùng nếu không lệnh thực hiện xem như bị lỗi.

4.1 Sử dụng ký tự *

Ký tự * đại diện cho 0 hoặc nhiều ký tự bất kỳ nào đó. Có nghĩa là "bất cứ cái gì ở đây".
Ví dụ:
/etc/g* sẽ trùng với tất cả các tập tin trong thư mục /etc mà chúng có tên bắt đầu bằng ký tự **g**.
/tmp/my*1 trùng với tất cả các tập tin trong thư mục /tmp mà chúng có tên bắt đầu với **my** và kết thúc với **1**.

4.2 Sử dụng ký tự ?

Ký tự **?** đại diện cho bất kỳ một ký tự nào đó.
Ví dụ: **myfile?** sẽ trùng với tất cả các tập tin mà tên bao gồm **myfile** và theo sau bằng một ký tự bất kỳ
/tmp/notes?txt sẽ trùng với cả hai tập tin **/tmp/notes.txt** và **/tmp/notes_txt**, nếu chúng tồn tại.

4.3 Ký tự []

Ký tự này cũng giống như ký tự **?**, nhưng có một số điểm đặc biệt hơn. Để sử dụng ký tự này, hãy để bất kỳ ký tự nào mà bạn muốn trùng khớp vào bên trong **[]**. Kết quả của biểu thức sẽ là một ký tự mà nó trùng khớp với bất kỳ một ký tự nào trong cặp **[]** Bạn cũng có thể dùng dấu – để mô tả khoảng ký tự sẽ trùng khớp.
Ví dụ:
myfile[12] sẽ trùng khớp với myfile1 và myfile2.
[Cc]hange[Ll]og sẽ trùng khớp với Changelog, ChangeLog, changeLog, và changelog. **ls /etc/[0-9]*** sẽ liệt kê tất cả các tập tin trong thư mục /etc mà chúng bắt đầu với một số. **ls /tmp/[A-Za-z]*** sẽ liệt kê tất cả các tập tin trong thư mục /etc mà chúng bắt đầu với một ký tự hoa hay thường.

4.4 Ký tự [!]

Ký tự **[!]** tương tự như cấu trúc **[]** , ngoại trừ thay vì trùng khớp bất kỳ ký tự nào nằm trong cặp dâu [], nó sẽ trùng khớp với bất kỳ ký tự nào mà **không** được liệt kê giữa **[!** và **]**.
Ví dụ: **rm myfile[!9]** sẽ xóa tất cả các tập tin có tên bắt đầu là **myfile** theo sau là một ký tự không phải là ký tự 9 (tức ngoại trừ tập tin **myfile9**).

4.5 Wildcard caveats

Bởi vì **bash** xử lý các ký tự đại diện (**?**, **[**, **]**, ***) một cách đặc biệt, bạn cần cẩn thận khi nhập đối số của các lệnh có chứa các ký tự đặc biệt này. Ví dụ nếu bạn muốn tạo một tập tin có chứa chuỗi **[fo]***, lệnh sau đây có thể thực hiện công việc mà bạn không trong đợi:

```
$ echo [fo]* > /tmp/mynewfile.txt
```

Nếu mẫu **[fo]*** trùng với bất kỳ tập tin nào trong thư mục hiện hành, khi đó bạn sẽ thấy tên của các tập tin này trong nội dung của tập tin /tmp/mynewfile.txt hơn là chuỗi **[fo]*** như mong đợi. Có thể khắc phục tình trạng trên bằng cách bao xung quanh các ký tự

đặc biệt dấu nhấy đơn ’ để bảo với **bash** xử lý các ký tự đặc biệt như những ký tự thông thường:

$ echo '[fo]*' > /tmp/mynewfile.txt

Trong ví dụ trên, tập tin mới tạo ra sẽ chứa chuỗi **[fo]*** như ta mong đợi.

Hoặc có thể dùng ký tự ‘\’ đi liền phía trước ký tự đặc biệt để yêu cầu **bash** xử lý chúng như các ký tự bình thường:

$ echo \[fo\]* > /tmp/mynewfile.txt

Cả hai giải pháp trên đều hoạt động rất tốt.

Bài tập

Bài 1. Thực hiện và khảo sát lại tất cả các ví dụ trên

Bài 2. Thực hienj lệnh chuyển thư mục theo thứ tự sau:

- Chuyển về thư mục gốc
- Chuyển đến thư mục /bin
- Chuyển đến thư mục /usr/local/bin
- Chuyển ra thư mục trên một mức (local)
- Chuyển về thư mục người dùng

Bài 3. Thực hiện tạo Script có tên hello đơn giản trong thư mục /usr như sau:

$cat >hello
$echo “Xin chao!!!”
$^D (bấm ctrl + D)
$chmod 755 hello

Đứng tại thư mục thực hiện shell này dùng cú pháp: sh hello

Thực hiện chuyển thư mục về thư mục cá nhân và thực hiện shell trên với cú pháp: sh hello

Shell hello có thực hiện được không? Vì sao?

Bài 4. Sử dụng đường dẫn tuyệt đối để thực hiện shell hello

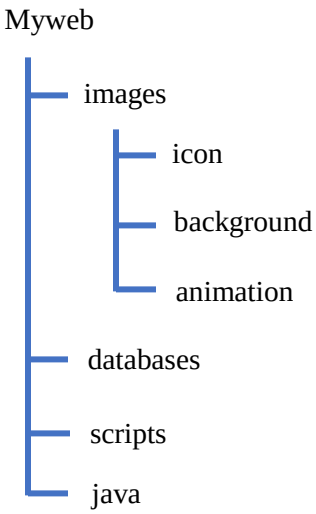
Bài 5. Xem nội dung của biếng PATH hiện tại của hệ thống. Thêm đường dẫn vào biến PATH shell hello (câu 3) có thể được thực hiện cú pháp: sh hello cho dù đang đứng ở bất kỳ vị trí nào?

Bài 6. Thực hiện lệnh ls -l trên hư mục /bin. Xác định các quyền của người sở hữu, nhóm và người dúng khác trên file ls và sh?

Thực hiện lệnh ls -l trên thư mục gốc. Xác định quyền và những người dùng hay nhóm người dùng tương ứng của thư mục root?

Bài 7. Thực hành lệnh file bằng cách tìm các file dạng ASCII và dùng lệnh less xem nội dung của file đó.

Bài 8. Tạo cây thư mục như sau trong thư mục cá nhân



Dùng lệnh tree để xem cây thư mục vừa tạo.

Bài 9: Tìm một số file có phần mở rộng .html và .class trong hệ thống file và thực hiện copy vài file .html vào thư mục Myweb và vài file .calss vào thư mục Myweb/scripts.

Bài 10: Thực hiện lệnh mv để di chuyển các file .class trong thư mục Myweb/scripts vào thư mục Myweb/java.

Bài 11: Thực hiện lệnh xóa các thư mục: Myweb/scripts, Myweb/images/animation.

Bài 12: Thiết lập quyền truy xuất cho thư mục Myweb/databases sao cho chỉ có người dùng root có toàn quyền và các người dùng khác không có bất kỳ quyền nào trên nó.

Bài 13: Thiết lập quyền trên thư mục trên thư mục Mywed/java sao cho người dùng có toàn quyền, các người dùng còn lại chỉ có quyền thực thi.